

# Go-ing Fast with MCP

Rapid Development of Custom MCP Servers  
for Security Agents



Matt Franz, Pydantic  
***DC State of the Stack 2026***



# During the next five minutes you will learn...

Why you should consider writing a custom MCP server for data-intensive platforms (SIEM, EDR, observability)

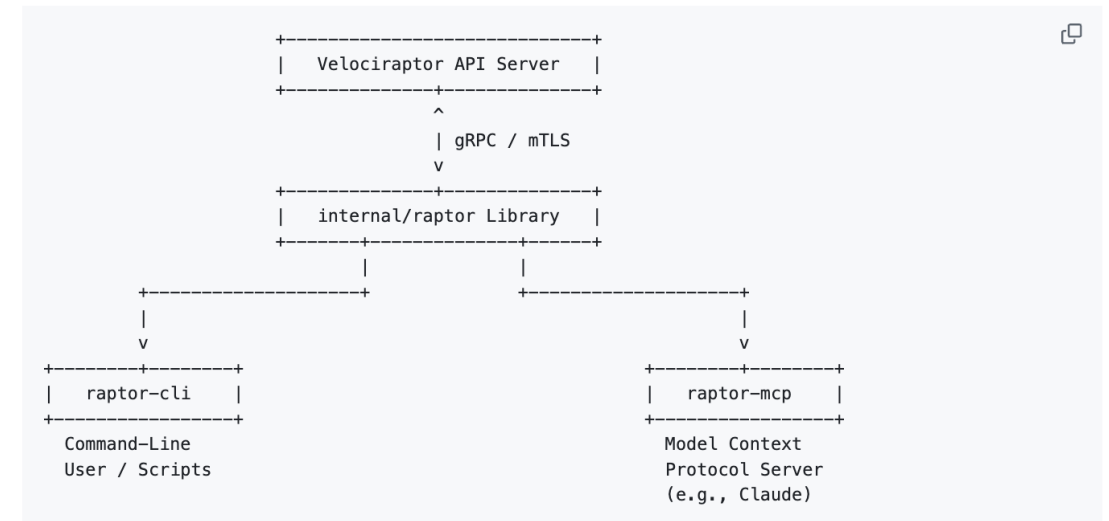
How to *iteratively* build, test, and run STDIO MCP servers (and CLIs) for Python agents or Claude, Codex, OpenCode, and more...

README

## go-velociraptor-mcp

A Go-based implementation of a Velociraptor command-line interface ( `raptor-cli` ) and a Model Context Protocol (MCP) server ( `raptor-mcp` ) for remote endpoint visibility, forensics, and orchestration via Velociraptor's gRPC/mTLS API.

### Architecture Overview



Both binaries are powered by a shared backend package ( `internal/raptor` ) that manages mTLS configuration parsing, connection pooling, raw VQL queries, parameter marshaling, and safety validation.

# Who am I?

---

*25+ years in Security and Infrastructure roles (IC to Director-level)*

Currently Security & Compliance Lead at Pydantic

For the last 2.5 years, I've been building blue-team security agents

In 2026, I've built over a dozen MCP servers in Go, Python, and Rust

Love European/Canadian travel and read way too much military history (was SIGINT Analyst)



# MCP and its Discontents

---

**Token burn** — large JSON payloads overwhelming agent context window

**Agent thrashing** — hallucinated queries and API calls leading to agent delays

**Security** — inherent vulnerabilities, RBAC challenges, and more

## *Alternative approaches:*

- CLIs called by coding agents
- Python code generation – processing and REST APIs

## *Where I find MCP still adds value:*

- Optimized tool calls (and descriptions) and input/output query guardrails to monitor and proxy agent interactions

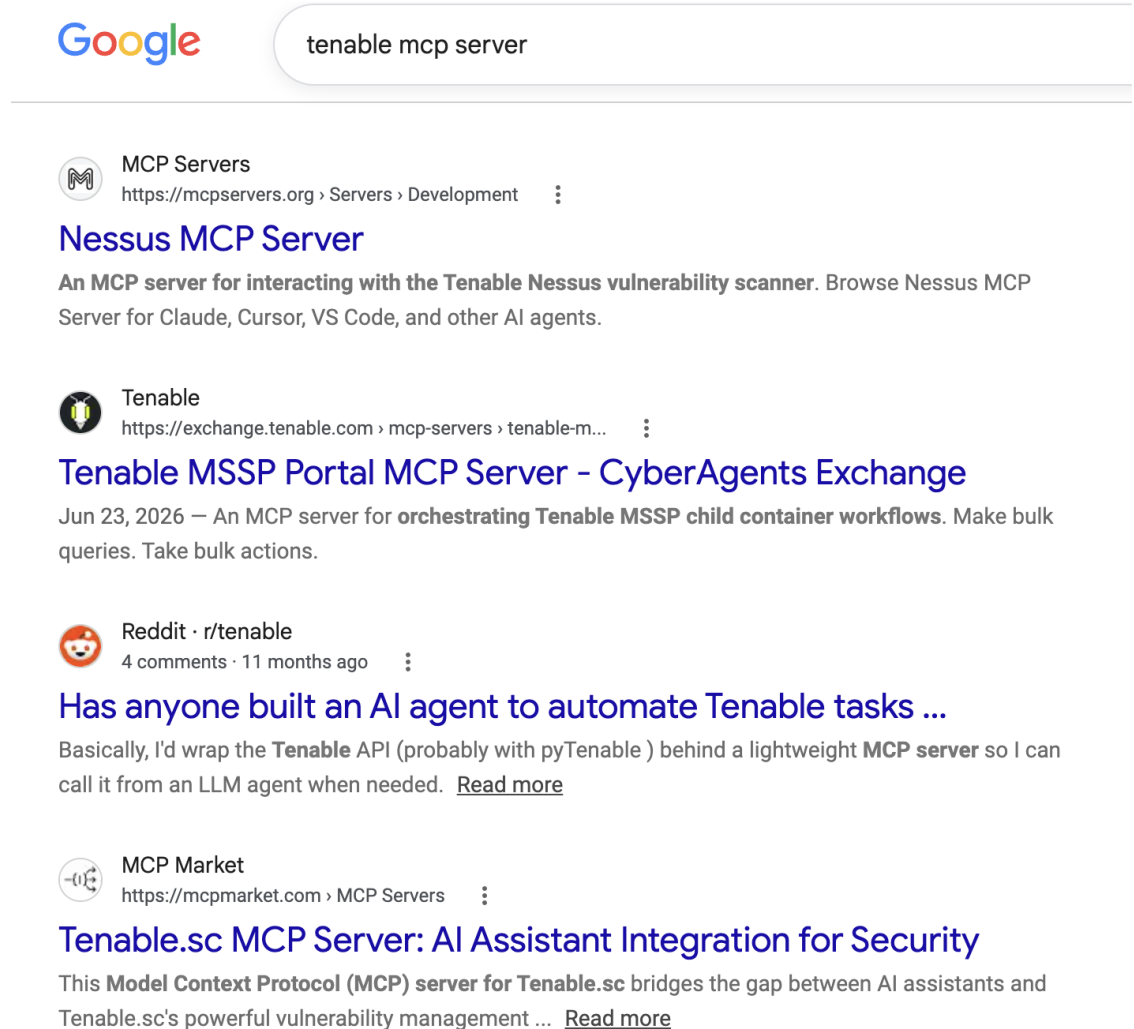


# Why build custom MCP servers?

## Issues using off the shelf MCP Servers

- Minimal logging or caching — often black boxes.
- Generic tool calls — tuned for everyone, not your telemetry or workflows
- Sketchy MCP marketplaces — do you really trust them with your SIEM API keys?

You probably need to learn the APIs (and data) to effectively create prompts, anyway...



The screenshot shows a Google search interface with the query 'tenable mcp server'. The search results are as follows:

- MCP Servers**  
https://mcpservers.org › Servers › Development  
**Nessus MCP Server**  
An MCP server for interacting with the Tenable Nessus vulnerability scanner. Browse Nessus MCP Server for Claude, Cursor, VS Code, and other AI agents.
- Tenable**  
https://exchange.tenable.com › mcp-servers › tenable-m...  
**Tenable MSSP Portal MCP Server - CyberAgents Exchange**  
Jun 23, 2026 — An MCP server for **orchestrating Tenable MSSP child container workflows**. Make bulk queries. Take bulk actions.
- Reddit · r/tenable**  
4 comments · 11 months ago  
**Has anyone built an AI agent to automate Tenable tasks ...**  
Basically, I'd wrap the **Tenable** API (probably with pyTenable ) behind a lightweight **MCP server** so I can call it from an LLM agent when needed. [Read more](#)
- MCP Market**  
https://mcpmarket.com › MCP Servers  
**Tenable.sc MCP Server: AI Assistant Integration for Security**  
This **Model Context Protocol (MCP) server for Tenable.sc** bridges the gap between AI assistants and Tenable.sc's powerful vulnerability management ... [Read more](#)

# Why Go?

---

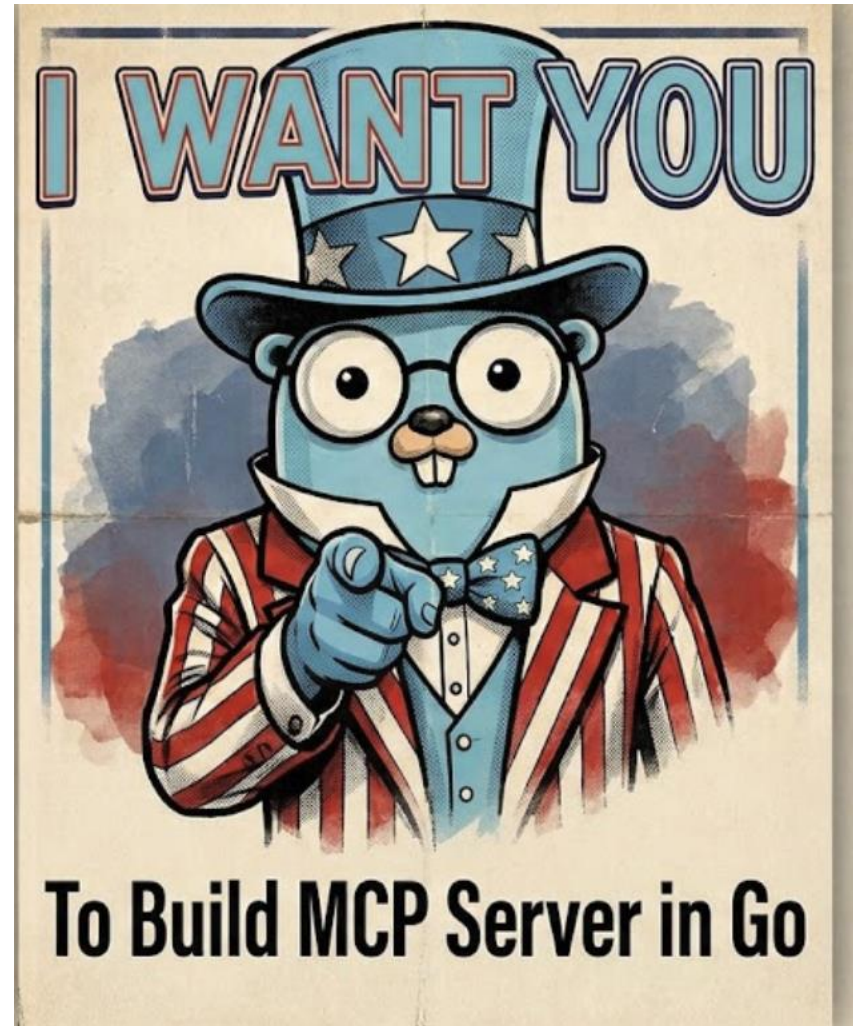
Increased supply-chain risks

Bloated Python/JS dependencies have slower startup-time

Go compiles to as a single cross-platform executable – choose features you bund!

Faster iteration and simplicity than Rust—especially for Python developers.

Mature, cross-platform libraries with lots of options



# A Go template

---

## Library Selection

- Official Go SDK (<https://github.com/modelcontextprotocol/go-sdk>)
- Use SPF13 (Cobra/Viper/pflag) for command-line handling

## Code Layout

```
internal/app/  
  config.go  
  client.go  
  schemas.go  
  results.go
```

```
cmd/app-mcp/  
  main.go  
  tools.go
```

```
cmd/app-cli/  
  main.go
```



# Lessons — start with the CLI

---

Build the CLI first — reuse a client library, or generate from Swagger/OpenAPI (worst case: HTML/PDF)

Explore the data — list / get / describe

Start with 3–5 tools — enough to learn the shape and the use cases

# Lessons — instrument and iterate

---

Instrument from day one — logging, observability, test coverage

Add an LLM harness — Pydantic AI against common queries

Explore, then specialize — let telemetry tell you the next tool to build

# Testing layers

---

**Unit tests** — Both for CLI and MCP server

**Direct JSON-RPC** smoke test — no LLM required

**Pydantic AI** harness — real tools with a range of models.

**Exploratory** with Claude, Codex, OpenCode, etc.

# The tool ladder

---

| Intent               | Tool           | Why                |
|----------------------|----------------|--------------------|
| See the surface      | list_tables    | Cheapest inventory |
| Find relevant data   | search_tables  | Names + columns    |
| Inspect one schema   | describe_table | No row payloads    |
| Learn shape + values | preview_table  | Schema + sample    |
| Query one table      | query_table    | Validated, bounded |
| Join or advanced SQL | run_query      | Flexible, riskiest |

Prefer the narrowest tool that answers the question.

## Tool typing — args

---

```
type SearchProcessesArgs struct {  
    Executable  string `jsonschema:"Exact path match. No wildcards."`  
    CommandLine string `jsonschema:"Token search (OR), not substring."`  
    HashSHA256  string `jsonschema:"SHA256 for malware detection."`  
    // + ProcessName, Host, Start/End  
}
```

# Guard Out — bound the query

Start read-only and **restrict index and scope** to prevent expensive, unbounded scans

Use **aggregation queries** with sane defaults to **get the shape of the data**

Require filters — and always bound the time range (and mind storage tiers)

**Per-tool timeouts** — surface partial results, relative to data you need

Implement **disk/memory caching** in the MCP server



# Guard In — bound the response

---

Cap rows and payloads returned to the LLM

Provide helpful error message to inform agent retry

Export separately — paginate to files, never bulk JSON into context, MCP supports async and progress

# E2E testing with Pydantic AI

Test across multiple  
Providers and Models

Normalize model queries  
and catch hallucinations

Test tool descriptions

Easy integration with Logfire  
for token, turn, and  
exception monitoring

```
SYSTEM_PROMPT = """
You are an Elastic Security expert.
Investigate Elasticsearch
```

```
Prefer the narrow, fast-path over the
cached history lookups, search for
search_security_alerts for a specific
broader network/endpoint etc.
else fits. Tool results prefixed with a cache indicator mean the answer was
served from (or written to) cache rather than freshly queried.
```

```
CRITICAL: After using tools, always provide a detailed final answer that includes:
1. Summary of what you discovered or found
2. Which specific tools you used and why each one
3. Key findings from the data (include sample rows if relevant)
4. Your analysis and conclusions
```

```
transport = StdioTransport(command=server_path, args=[], env=dict(os.environ))
async with MCPToolset(transport, max_retries=3) as server:
    agent = Agent(
        model_name,
        toolsets=[server],
        capabilities=[hooks],
        system_prompt=SYSTEM_PROMPT,
    )

    tasks = [*TASKS, build_export_task()]

    try:
        for title, prompt in tasks:
            stats.reset_task()
            start = time.perf_counter()
            logger.info("\n" + "=" * 20)
            logger.info(f" TASK: {title}")
            logger.info("=" * 20)
```

# Implement observability early

---

Logs — one agent (Claude) builds, another (Codex) observes and gives feedback

Traces — exceptions and spans

Metrics — query latency

# Agentsview

---

Local agentic analytics for Claude, Codex, and OpenCode sessions

Tool sequence, arguments, and outcomes per task

<https://github.com/kenn-io/agentsview>

# Logfire

---

OpenTelemetry traces across model calls, tool calls, and spans

Queryable after the fact, with a built-in MCP server

<https://pydantic.dev/logfire>

# For more info

**osqueryi-mcp** — starter server, smoke harnesses, framework examples

**elastic-security-mcp** — structured tools, raw DSL fallback, caching, export

**mcp-threat-hunt-aug-2026** — the long-form companion talk

<https://github.com/mdfranz/dc-state-of-stack-2026-go-mcp>

